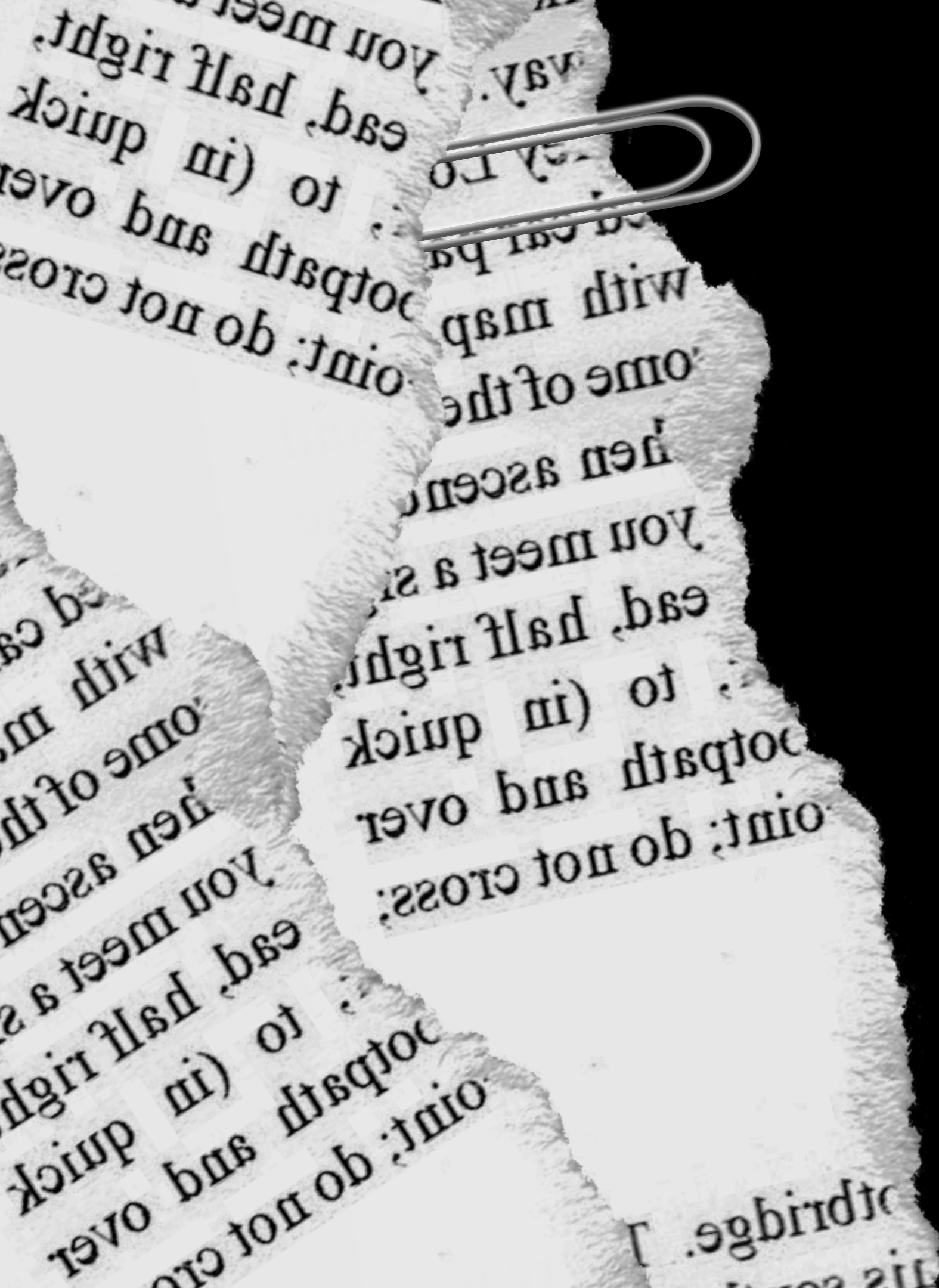


ADRIÁN REY MONTEAGUDO

MUSIC IPFS STUDIO

MUSIC IPFS STUDIO

A torn piece of paper with a paperclip is positioned on the left side of the image. The paper has a rough, deckled edge and contains text that is mirrored across its vertical axis. The text is in a serif font and appears to be a mix of English and Spanish. The paperclip is a simple metal wire clip, holding the paper in place. The background is a solid black color.

ÍNDICE

- Introducción
- Arquitectura
- Funciones principales
- Funcionamiento
- Conclusión

INTRODUCCIÓN

INTRODUCCIÓN

Problema

La industria musical digital es centralizada. Los artistas pierden control sobre la propiedad, el reparto de ingresos y la trazabilidad de sus ventas

Solución

Music IPFS Studio es un ecosistema descentralizado que usa Web3 (Ethereum + IPFS) para permitir que los músicos conserven la propiedad y los beneficios de su obra sin intermediarios.

Objetivos

Implementar almacenamiento distribuido con IPFS, crear NFTs musicales, y desarrollar un marketplace con un sistema de royalties automáticos usando ERC-2981





ARQUITECTURA

ARQUITECTURA

Frontend React

Desarrollada con React (App.js), utiliza Ethers.js para interactuar con la wallet (connectWallet) y los Smart Contracts

Blockchain

Contiene los contratos ArtNFT (propiedad) y Marketplace (comercio)

IPFS

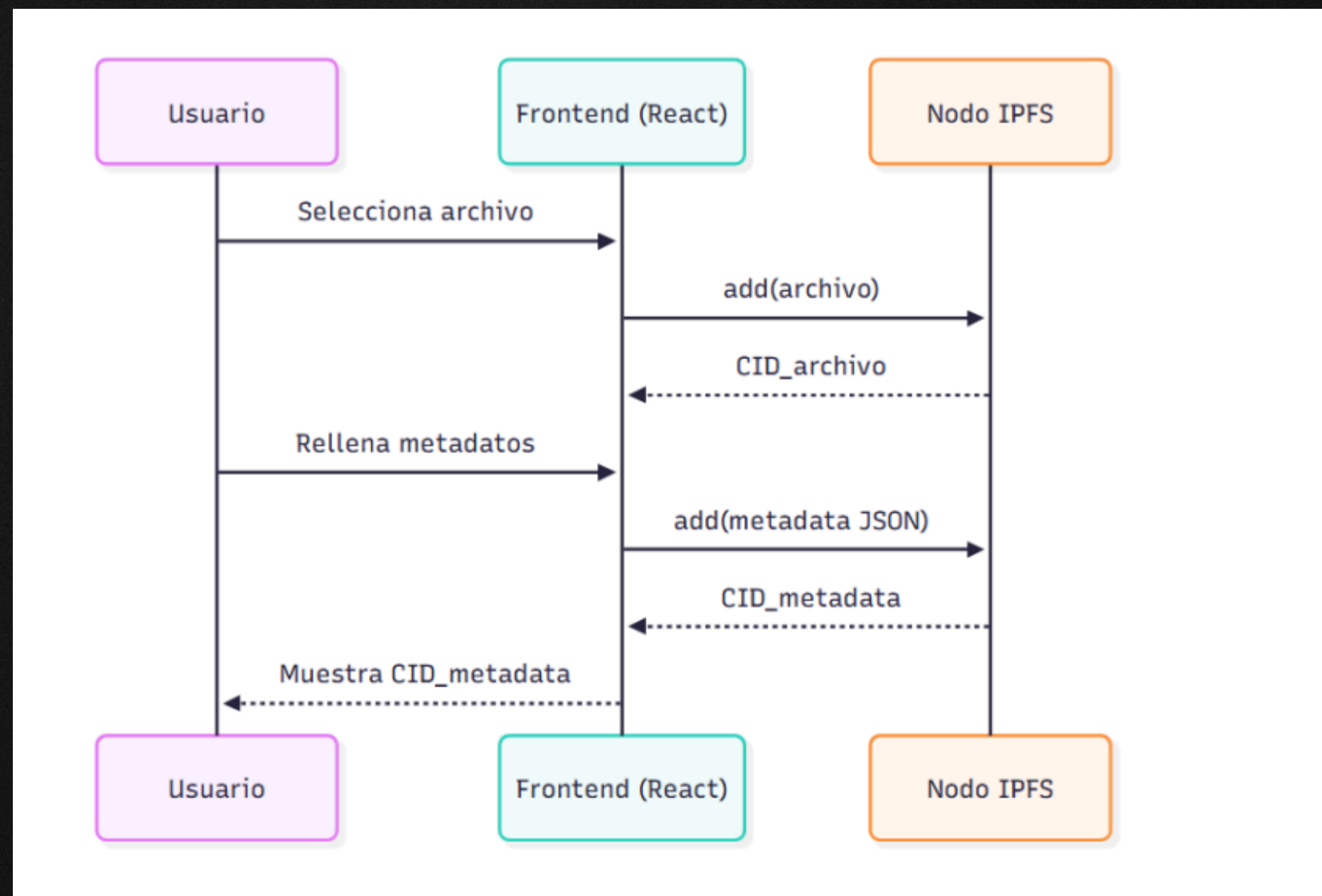
Almacena de forma descentralizada los archivos musicales y los metadatos (JSON)

Seguridad

Los contratos emplean librerías auditadas de OpenZeppelin (ERC721URIStorage, ERC2981, Ownable). Específicamente, el Marketplace utiliza ReentrancyGuard para ser nonReentrant y prevenir ataques de reentrada

FLUJO DE CREACIÓN

FLUJO DE CREACIÓN



```
// Subida archivo IPFS
const uploadMusicFile = async () => {
  if (!musicFile) {
    alert("Selecciona un archivo primero");
    return;
  }

  try {
    const client = await create('/ip4/127.0.0.1/tcp/5001');

    const arrayBuffer = await musicFile.arrayBuffer();
    const buffer = Buffer.from(arrayBuffer);

    const result = await client.add(buffer);
    const cid = result.cid.toString();
    setUploadedCid(cid);

    alert("Archivo subido correctamente a IPFS");
  } catch (err) {
    console.error(err);
    alert(err.message);
  }
};
```

```
// Subida metadata JSON IPFS
const uploadMetadata = async () => {
  if (!uploadedCid) {
    alert("Sube un archivo primero");
    return;
  }

  if (!metadata.title || !metadata.genre) {
    alert("Rellena título y género");
    return;
  }

  try {
    const client = await create('/ip4/127.0.0.1/tcp/5001');

    const creator = account || await getAccountAddress();

    const metadataJson = {
      title: metadata.title,
      genre: metadata.genre,
      instruments: metadata.instruments.split(",").map(i => i.trim()),
      bpm: parseInt(metadata.bpm) || 0,
      author: creator,
      audio: `ipfs://${uploadedCid}`,
      createdAt: Date.now()
    };

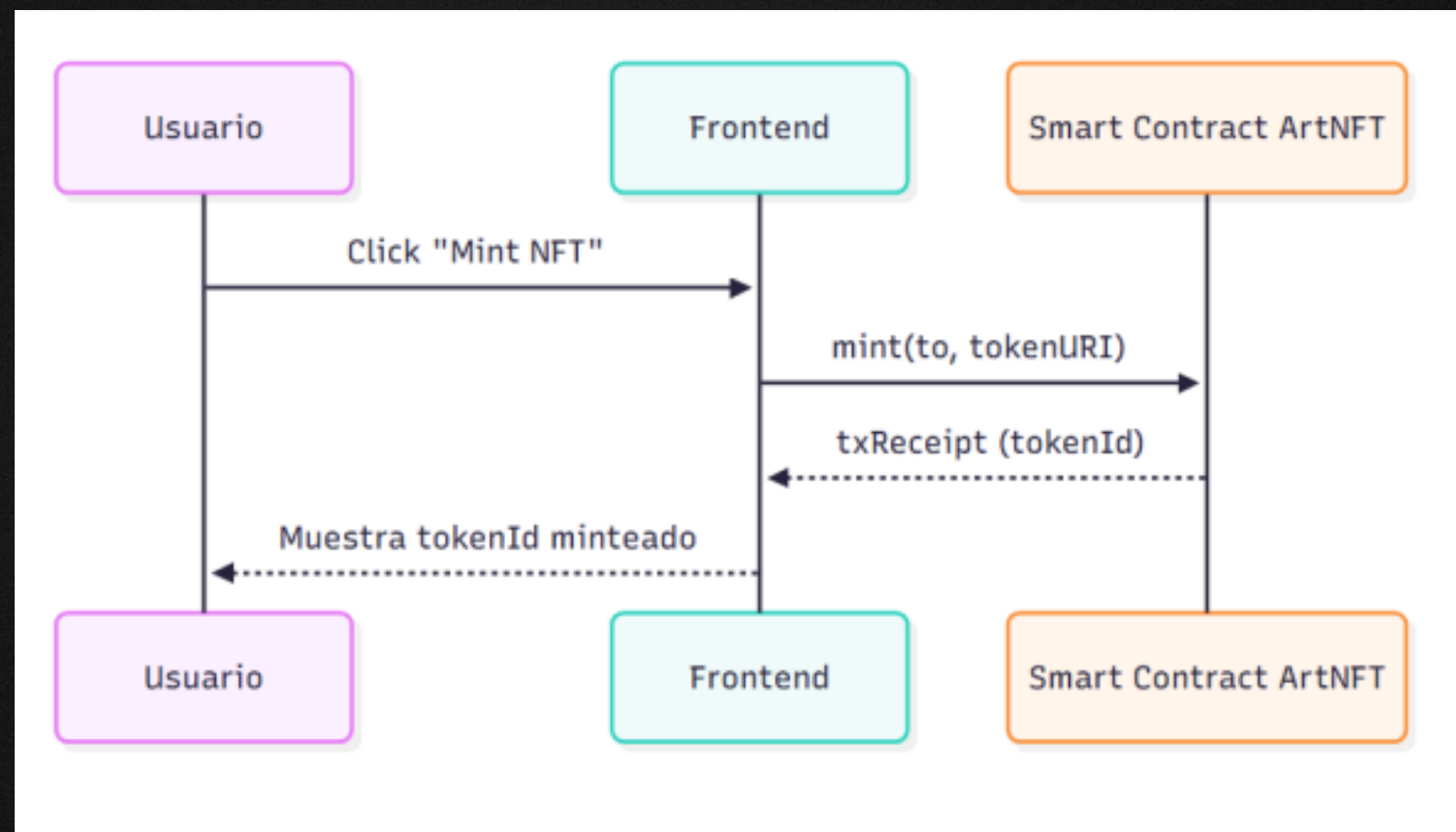
    const buffer = Buffer.from(JSON.stringify(metadataJson));
    const result = await client.add(buffer);
    const cid = result.cid.toString();

    console.log("DEBUG - Metadata CID generado:", cid);

    setMetadataCid(cid);
    alert("Metadata subida correctamente");
  } catch (err) {
    console.error(err);
    alert(err.message);
  }
};
```

MINTEO NFT

MINTEO NFT



```
/// MINT con royalty
function mint(
    address recipient,
    string memory tokenURI,
    address creator,
    uint96 royaltyBps // ej: 500 = 5%
) external onlyOwner returns (uint256) {
    uint256 tokenId = _nextTokenId;
    _nextTokenId++;

    _safeMint(recipient, tokenId);
    _setTokenURI(tokenId, tokenURI);

    // registrar autor original
    mintData[tokenId] = MintInfo(creator);

    // fijar royalty
    _setTokenRoyalty(tokenId, creator, royaltyBps);

    emit Minted(recipient, tokenId, tokenURI);
    return tokenId;
}
```

```
async function mintNFT() {
    if (!metadataCid) {
        alert("Primero sube la metadata para generar el tokenURI");
        return;
    }
    if (!account) {
        alert("Conecta la wallet antes de mintear");
        return;
    }

    try {
        // Comprueba que la ABI esté presente
        if (!abis || !abis.ArtNFT) {
            throw new Error("ABI de ArtNFT no encontrada (revisa src/contracts.js).");
        }

        const signer = getSigner();
        const artNFT = getArtNFTContract(signer);

        const tokenURI = `ipfs://${metadataCid}`;
        const royaltyBps = 500; // Regalías del 5% (500 basis points)

        console.log("Minting NFT with tokenURI:", tokenURI);

        // Llamar a la función mint con los 4 argumentos requeridos
        let tx = await artNFT.mint(account, tokenURI, account, royaltyBps);
        let receipt = await tx.wait();
    } catch (err) {
        console.error("Error en mintNFT:", err);
        const msg = err?.reason || err?.data?.message || err?.error?.message || err?.message || String(err);
        alert("Error durante el mint del NFT: " + msg);
    }
}
```

```
/// Extraer tokenId del evento Minted
let tokenId = null;
if (receipt && receipt.events) {
    const evt = receipt.events.find(e => e.event === "Minted");
    if (evt && evt.args) {
        tokenId = evt.args[1].toString(); // El segundo argumento del evento es el tokenId
    }
}

if (!tokenId) {
    console.warn("No se pudo extraer tokenId desde eventos. Receipt:", receipt);
    alert("Mint completado pero no se pudo obtener el tokenId del evento. Revisa la consola.");
    return;
}

console.log("NFT mintado con tokenId:", tokenId);
alert(`NFT mintado con éxito. Token ID: ${tokenId}`);

} catch (err) {
    console.error("Error en mintNFT:", err);
    const msg = err?.reason || err?.data?.message || err?.error?.message || err?.message || String(err);
    alert("Error durante el mint del NFT: " + msg);
}
}
```

COMPRA

COMPRA

```

async function buyNFT() {
  try {
    if (!itemToBuy) return alert("NFT no seleccionado.");

    const signer = getSigner();
    const marketplace = getMarketplaceContract(signer);

    const priceWei = itemToBuy.priceWei;

    console.log("Comprando NFT...");
    const tx = await marketplace.buy(itemToBuy.listingId, { value: priceWei });
    await tx.wait();

    alert("🎉 ¡Compra realizada con éxito!");

    setShowBuyModal(false);

    // Actualizar datos
    loadMarketplaceItems(); // Actualiza los items del marketplace
    await loadUserNFTs(); // Actualiza los NFTs del usuario comprador
  } catch (err) {
    console.error("Error al comprar NFT:", err);
    alert("Error en la compra: " + err.message);
  }
}

```

```

function buy(uint256 listingId) external payable nonReentrant {
  Listing storage l = listings[listingId];
  require(l.active, "Not active");
  require(msg.value == l.price, "Incorrect value");

  l.active = false;

  uint256 fee = (msg.value * feeBasis) / 10000;
  uint256 afterFee = msg.value - fee;

  // pagar fee
  if (fee > 0) {
    (bool sentFee, ) = payable(feeRecipient).call{value: fee}("");
    require(sentFee, "Fee transfer failed");
  }

  // obtener royalties ERC2981
  (address royaltyReceiver, uint256 royaltyAmount) =
    IERC2981(l.nftAddress).royaltyInfo(l.tokenId, msg.value);

  uint256 sellerPayment = afterFee - royaltyAmount;

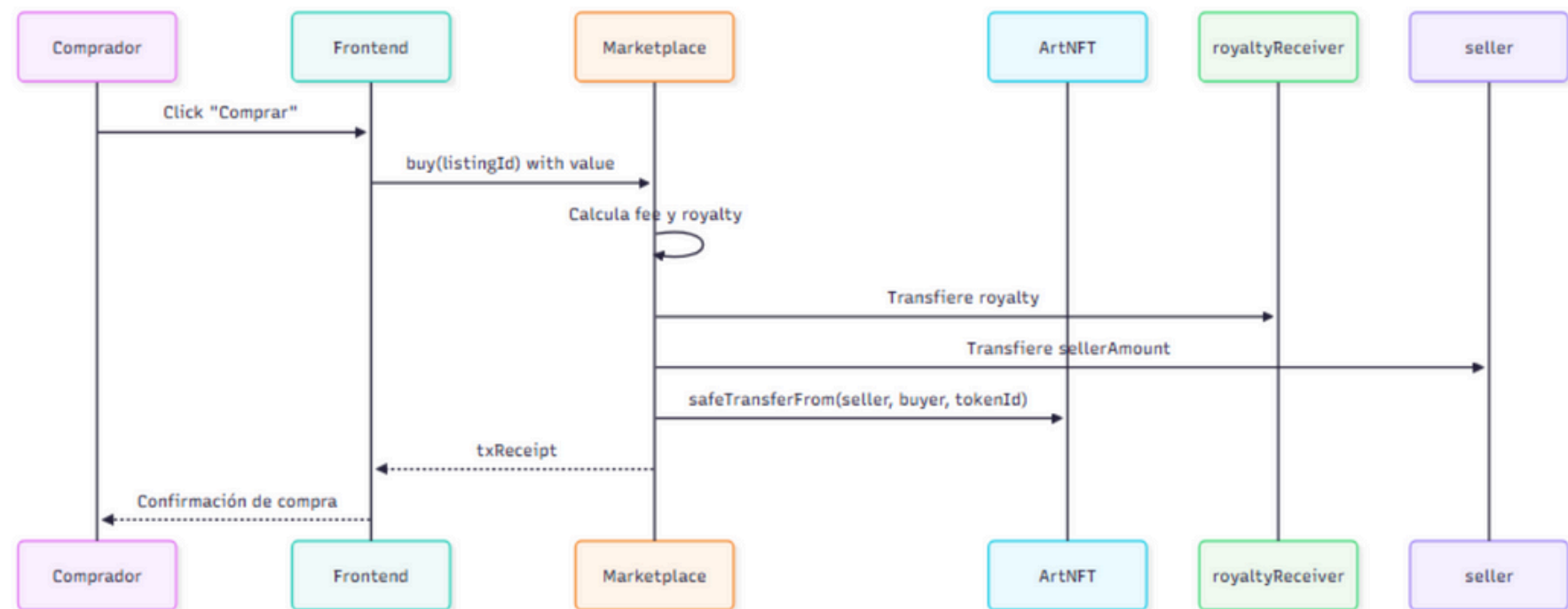
  // pagar royalties
  if (royaltyAmount > 0) {
    (bool sentRoyalties, ) = payable(royaltyReceiver).call{value: royaltyAmount}("");
    require(sentRoyalties, "Royalty payment failed");
  }

  // pagar al vendedor
  (bool sentSeller, ) = payable(l.seller).call{value: sellerPayment}("");
  require(sentSeller, "Seller transfer failed");

  // transferir NFT
  IERC721(l.nftAddress).safeTransferFrom(l.seller, msg.sender, l.tokenId);

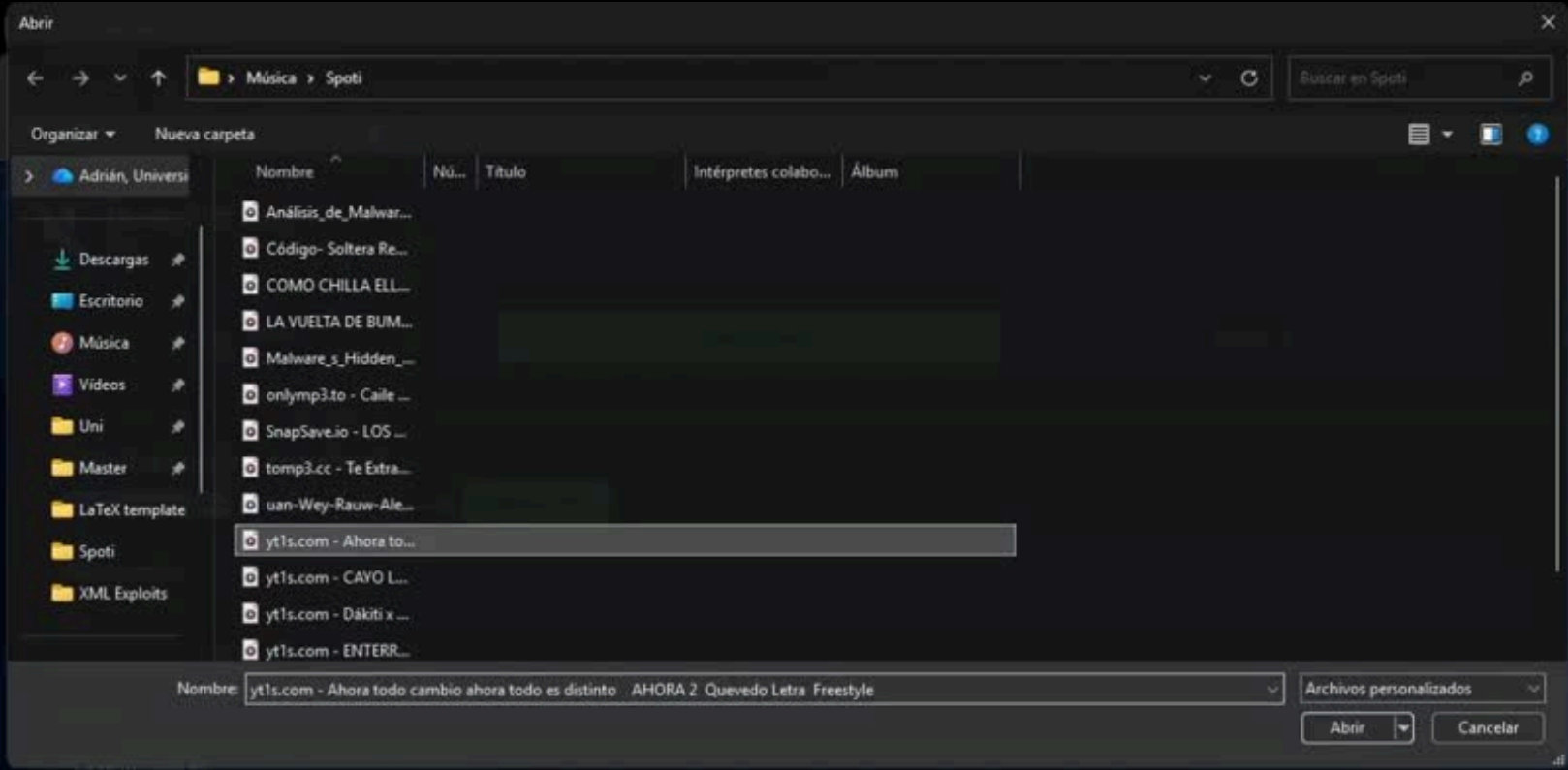
  emit Bought(listingId, msg.sender, msg.value);
}

```



FUNCIONAMIENTO

FUNCIONAMIENTO





CONCLUSIÓN

NOISN7ONOC

Music IPFS Studio demuestra la viabilidad de la distribución musical descentralizada, proporcionando trazabilidad, propiedad verificable y un modelo de negocio sostenible para los creadores gracias a los royalties automáticos

Se logra mayor control del artista, independencia de intermediarios y persistencia del contenido sin censura

ADRIÁN REY MONTEAGUDO

MUSIC IPFS STUDIO

MUSIC IPFS STUDIO